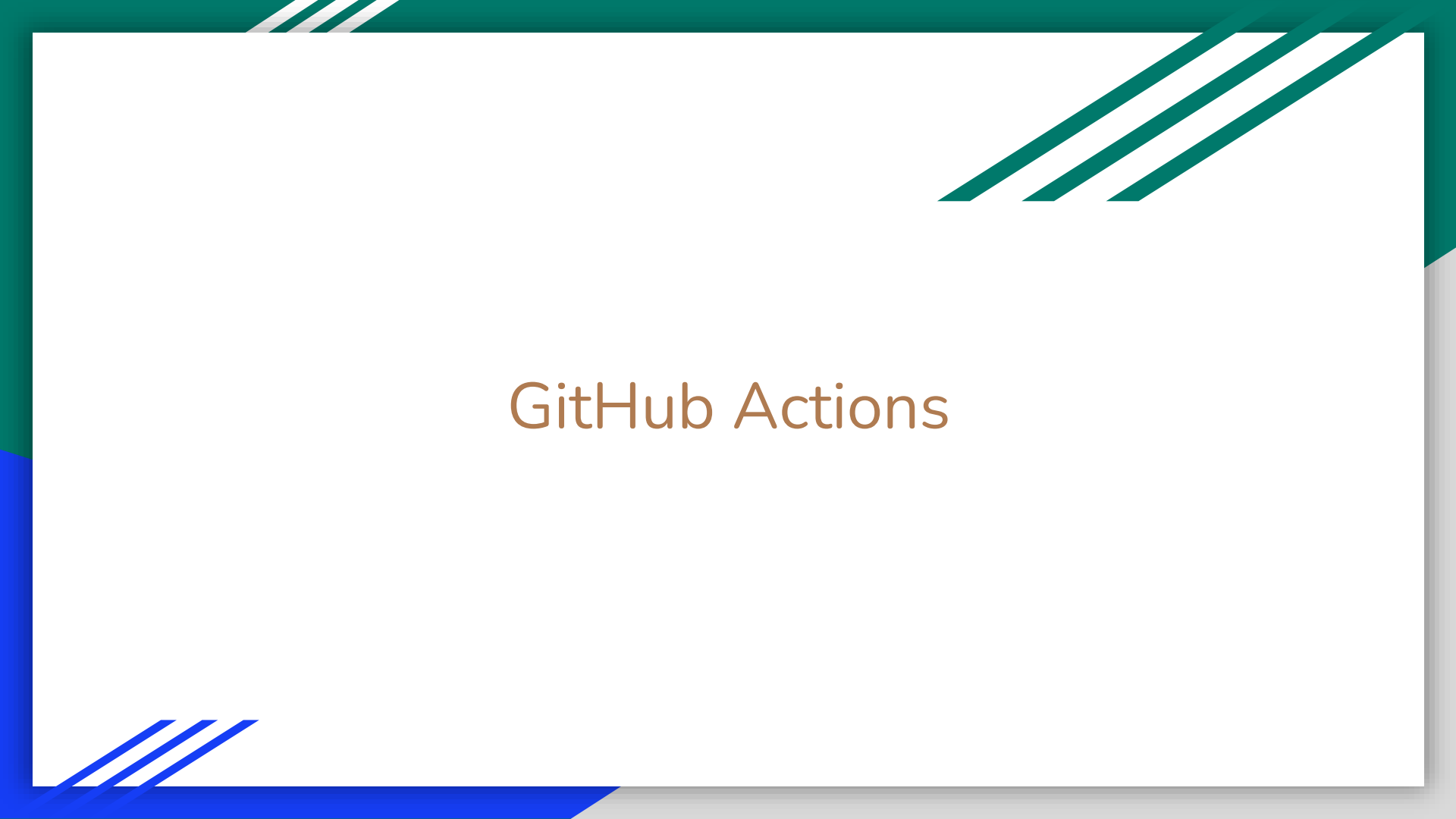




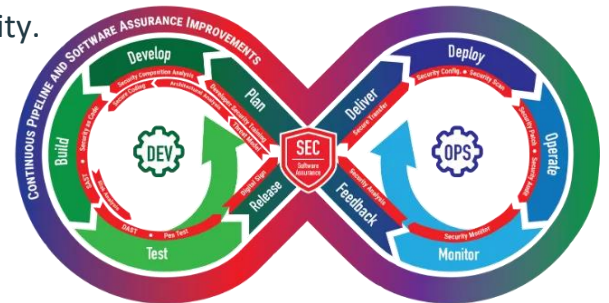
GitHub Actions

Introduction to CI/CD

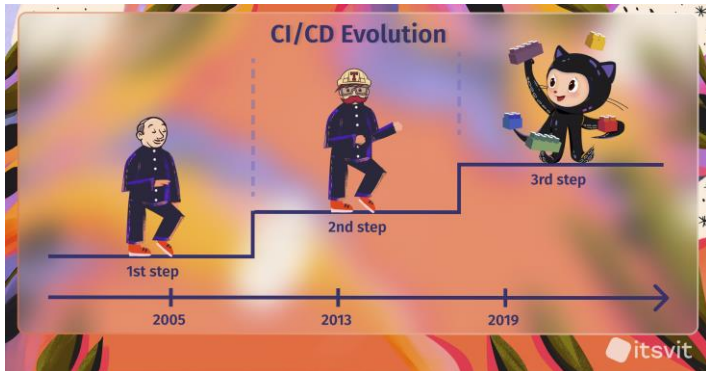
Continuous Integration (CI) and Continuous Deployment (CD) are cornerstones of modern software development that streamline the process from code commit to deployment. CI involves automating the integration of code changes from multiple contributors into a single software project. This process includes automated testing to detect errors quickly. CD automates the deployment of software to production, enabling fast and reliable delivery of features to users.

Benefits include:

- Faster release rate: Automate testing and deployment for quicker iteration.
- Higher quality: early bug detection and resolution improve software quality.
- Efficient collaboration: simplifies developer contributions and feedback.
- Reduced risk: smaller, incremental updates reduce deployment risk.

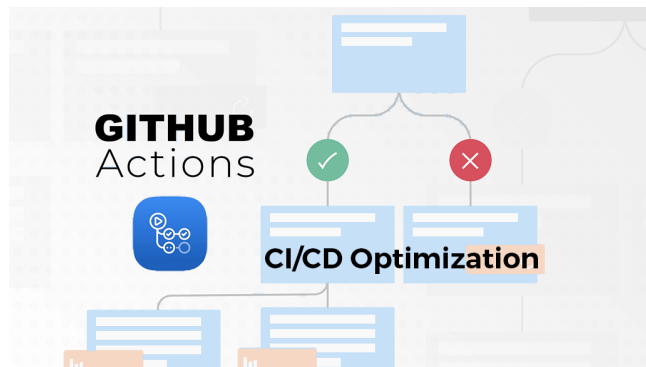


Evolution of CI/CD



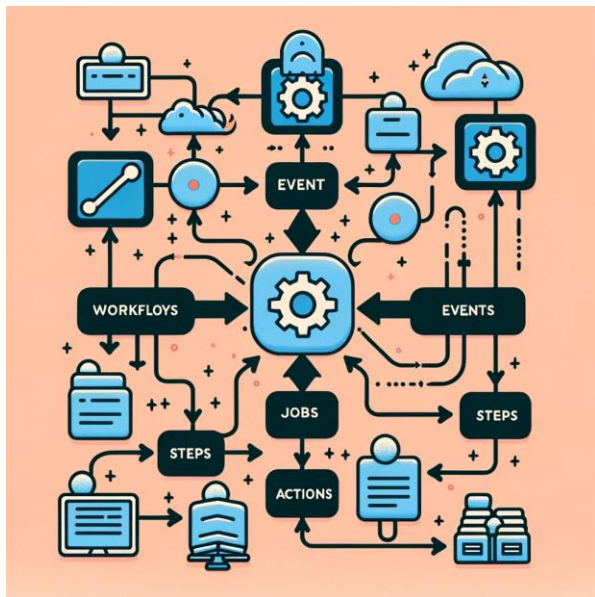
- Pre-CI era: Integration was manual, infrequent, and error-prone, leading to "integration hell."
- Introduction of CI: The concept of Continuous Integration emerged in the early 2000s, promoting regular code integration and automated testing to identify issues early.
- Advent of CD: Continuous Deployment followed, automating the deployment process to ensure quick and reliable delivery of code changes to production environments.
- Innovation and tools: The introduction of sophisticated CI/CD tools (like Jenkins, Travis CI, GitHub Actions) facilitated these practices, making them more accessible and customizable.
- Integration with Agile and DevOps: CI/CD practices became integral to Agile and DevOps methodologies, emphasizing rapid, efficient, and collaborative development cycles.

Introduction to GitHub Actions



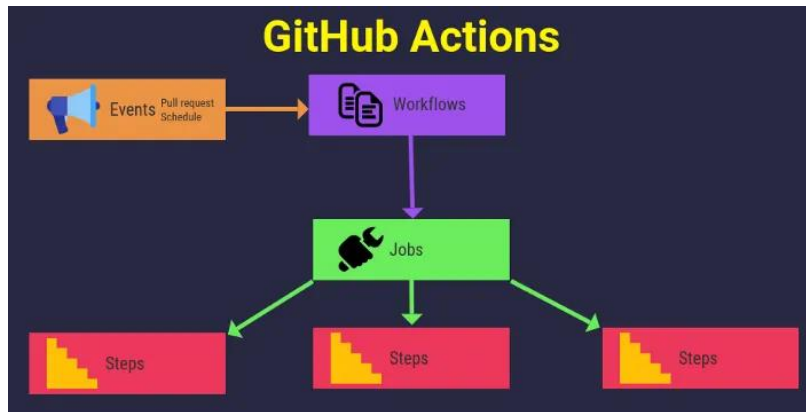
GitHub Actions is a powerful automation platform that integrates directly into GitHub, enabling developers to automate their software workflows directly from their repositories. Launched in 2018, it marked a significant evolution in the CI/CD ecosystem by making automation accessible within the same environment where code is stored and reviewed. With GitHub Actions, developers can create custom workflows to build, test, and deploy their code based on a variety of triggers, such as push events or pull requests. This integration streamlines the development process, reduces the potential for errors, and significantly speeds up the delivery of applications. Its significance lies in its ability to democratize automation for developers by providing a flexible, straightforward platform that supports a vast array of programming languages and frameworks, thereby enhancing the efficiency and reliability of software development processes across the globe.

The Basics of GitHub Actions



GitHub Actions enables automated workflows within your GitHub repositories. A workflow, defined in a YAML file, automates processes like building, testing, and deploying code. These workflows are triggered by events—specific activities in your repository such as a push or a pull request. Within each workflow, you define jobs, which are sets of steps that run on runners (virtual machines). Steps are individual tasks that can run commands or actions. Actions, the smallest units of a workflow, are reusable pieces of code that perform a specific job, such as setting up a development environment or deploying to a server. This framework allows developers to automate their software development processes, making them more efficient, consistent, and error-resistant.

Components of GitHub Actions



- **Workflow:** Central to GitHub Actions, it defines the automation process.
- **Trigger:** Specifies events that initiate workflows, like pushes or pull requests.
- **Job:** Represents a sequence of tasks to be executed, often in parallel or sequentially.
- **Step:** Individual task within a job, such as running a script or deploying code.
- **Action:** Reusable units of work defined in YAML files, allowing customization and integration.
- **Runner:** Environment where workflows execute, providing necessary resources and tools.
- **Artifact:** Data produced by a workflow, like build outputs or test results, often stored for future use or reference.

Workflows in Detail

Workflows are at the heart of GitHub Actions, serving as automated procedures that are defined in a repository to handle software development tasks. A workflow is composed of one or more jobs and is triggered by specific events, such as a push to a branch, a pull request, or on a schedule. Each workflow is described using YAML syntax in a `.github/workflows` directory of a GitHub repository. This setup allows for the automation of testing, building, and deploying code, depending on the sequence of tasks defined. Workflows can run jobs concurrently or sequentially and can be configured to run on different types of runners, including GitHub-hosted runners or self-hosted runners. By leveraging workflows, developers can significantly reduce manual intervention, ensuring consistent and efficient execution of development processes.

Events that Trigger Workflows

- **Code changes:** Events triggered by code commits or merges into the repository.
- **Pull Requests:** Initiate workflows when pull requests are opened, closed, or updated.
- **Issues:** Events related to issue creation, comments, or updates can trigger workflows.
- **Repository dispatch:** Custom events triggered externally to initiate workflows.
- **Scheduled events:** Workflows can be scheduled to run at specific times or intervals.
- **External events:** Integration with external services or systems can trigger workflows.
- **Manual triggers:** Workflows can be manually triggered by users via the GitHub interface.

Jobs and Parallelism

Jobs serve as units of work within workflows, each containing a set of steps to execute. By default, jobs run sequentially, one after another. However, workflows can be optimized for performance by running multiple jobs concurrently, a practice known as parallelism.

```
name: first workflow

on: [push]

jobs:
  first-job:
    runs-on: ubuntu-latest
    steps:
      - name: Listing contents before checkout
        run: |
          pwd
          ls

      - name: checkout code
        uses: actions/checkout@v3

      - name: Listing contents after checkout
        run: |
          ls
```

Steps: The Work Units

Each step represents an individual task that can run commands, scripts, or actions within a job. Steps are executed sequentially within the context of a job, allowing for a series of tasks to be performed in an ordered manner. This structure enables developers to build detailed processes, such as compiling code, running tests, or deploying applications, in a clear, step-by-step workflow. Steps can use actions, which are reusable pieces of code, to perform specific tasks, making workflows more modular and easier to maintain. By utilizing steps effectively, developers can create precise and efficient CI/CD pipelines tailored to their project's needs.

```
name: environments workflow
on: [push]

jobs:
  windows-job:
    runs-on: windows-latest
    steps:
      - name: using powershell
        shell: pwsh
        run: Get-Location
  macos-job:
    runs-on: macos-latest
    steps:
      - name: running in mac
        shell: bash
        run: echo Hello world
  linux-job:
    runs-on: ubuntu-latest
    steps:
      - name: running in ubuntu
        shell: bash
        run: echo Hello world
```

Actions: Reusable Code Chunks

Actions in GitHub Actions are reusable chunks of code that can be incorporated into steps within a job, streamlining workflow creation and maintenance. These actions enable developers to encapsulate frequently used commands, scripts, or utilities into standalone units that can be shared across multiple workflows or even with the GitHub community. By leveraging actions, developers can avoid redundancy, ensuring a DRY (Don't Repeat Yourself) approach to workflow design. This modularity not only accelerates the development of CI/CD pipelines but also promotes best practices and collaboration by allowing teams to use and contribute to a growing library of actions available in the GitHub Marketplace. Actions make workflows more efficient, customizable, and easier to update, embodying the principles of code reuse and automation.

Runners Explained

Runners in GitHub Actions are the servers that execute the jobs defined in workflows. Each runner can be a virtual machine hosted by GitHub or a self-hosted machine that you manage. Runners come equipped with a specific operating system, such as Linux, Windows, or macOS, allowing workflows to run in environments that match their deployment targets or testing requirements. They automatically download your code and execute the workflow's steps and actions. After a job is completed, the runner reports the results back to GitHub. This setup provides flexibility and control over the execution environment, ensuring that workflows can run in conditions that closely mimic production environments or meet specific project needs, thereby enhancing the reliability of the CI/CD process.

Setting Up Your First Workflow

Setting up your first workflow in GitHub Actions involves a few straightforward steps. Start by creating a `.github/workflows` directory in your repository. Inside this directory, create a YAML file (e.g., `main.yml`) to define your workflow. Begin the file with the name of your workflow and specify on which events it should trigger, such as on push or pull requests to the main branch. Next, define jobs that outline the tasks to be executed, breaking down these tasks into steps. Steps can include setting up your project environment, running tests, and deploying your code.

```
1 name: CI
2
3 on:
4   push:
5     branches:
6       - main
7
8 jobs:
9   build:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v2
13       - name: Run a one-line script
14         run: echo Hello, world!
15       - name: Run a multi-line script
16         run: |
17           echo Add other commands here
18           echo This is a multi-line script
```

YAML Syntax for GitHub Actions

YAML, or YAML Ain't Markup Language, is a human-readable data serialization standard used in GitHub Actions to define workflows. It allows developers to describe their automation processes, including triggers, jobs, steps, and actions, in a structured format. YAML files for GitHub Actions must be placed in the `.github/workflows` directory of a repository. Key elements include name for the workflow, on for specifying trigger events, jobs to outline tasks, and steps within those jobs, each possibly utilizing actions. Indentation is crucial in YAML to denote hierarchy and structure, with spaces typically used for indentation (tabs are not allowed). YAML's readability and simplicity make it an ideal choice for defining complex workflows in GitHub Actions, enabling developers to easily automate their CI/CD pipelines.

Using Secrets and Environment Variables

Secrets and environment variables in GitHub Actions allow you to manage sensitive data and configuration settings securely within your workflows. Secrets are encrypted variables stored at the repository or organization level, accessible in workflows but hidden from logs and not exposed to unauthorized users. Environment variables can be used to set configuration options and are defined in the YAML workflow file or in the repository settings. To use a secret in your workflow, you first add it to your repository's settings under "Secrets."

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to server
        run: deploy_script.sh
        env:
          SERVER_API_KEY: ${ secrets.SERVER_API_KEY }
```

Implementing Matrix Builds

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [10.x, 12.x, 14.x]
        os: [ubuntu-latest, windows-latest, macOS-latest]
    steps:
      - uses: actions/checkout@v2
      - name: Use Node.js ${ matrix.node-version }
        uses: actions/setup-node@v1
        with:
          node-version: ${ matrix.node-version }
      - run: npm install
      - run: npm test
      env:
        CI: true
```

Matrix builds in GitHub Actions enable you to run tests across multiple environments by defining a set of variables that generate a job matrix. This allows for parallel execution of jobs across different combinations of environments, such as operating systems, programming languages, or versions. The matrix strategy reduces the time required for testing by leveraging concurrency and ensures your code works as expected in diverse conditions.

Caching Dependencies

Caching dependencies in GitHub Actions can significantly reduce build times by storing and reusing parts of the workflow environment, such as libraries or packages, between runs. When a workflow runs, instead of downloading dependencies from scratch, the cache can be checked first. If the dependencies haven't changed, the workflow can use the cached versions, speeding up the process.

```
name: CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Cache node modules
        uses: actions/cache@v2
        with:
          path: ~/.npm
          key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }
          restore-keys: |
            ${ runner.os }-node-
```

Scheduling Workflows

- Scheduling workflows in GitHub Actions allows you to automate tasks to run at predetermined times, using cron syntax for scheduling. This feature is particularly useful for routine tasks like nightly builds, daily reports, or regular maintenance jobs. Scheduled workflows are defined within the `on` field of your workflow file, enabling precise control over when your workflows trigger, independent of code pushes or pull requests.
- This example sets up a workflow to run daily at 2 AM UTC. The cron syntax `'0 2 * * *'` specifies the minute (0), hour (2), day of the month (* for any), month (* for any), and day of the week (* for any), allowing for flexible scheduling to meet various automation needs.

```
on:  
  schedule:  
    - cron: '0 2 * * *' # Runs at 2 AM UTC every day
```

External Actions: Marketplace

The GitHub Marketplace is a central hub where you can find and use actions created by the GitHub community and beyond, enhancing your workflows without needing to reinvent the wheel. These actions cover a wide range of automation tasks, from code linting and security scanning to deployment. Using external actions simplifies workflow creation and extends GitHub Actions' capabilities with community-driven solutions.

```
name: CI

on:
  schedule:
    - cron: '0 2 * * *' # Runs at 2 AM UTC every day

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Cache node modules
        uses: actions/cache@v2
        with:
          path: ~/.npm
          key: ${ runner.os }-node-${ hashFiles('*/package-lock.json') }
          restore-keys: |
            ${ runner.os }-node-
```

Workflow Commands for Actions

```
echo "name=ACTION_STATE::success" >> $GITHUB_ENV
```

- Workflow commands for GitHub Actions enable actions to communicate with the GitHub Actions runner machine to set environment variables, output values, and more, directly from within scripts or actions. These commands follow a specific syntax, allowing for dynamic and flexible control over the workflow execution.
- In this example, a workflow command is used to set an environment variable `ACTION_STATE` to `success` within the action's execution context. This command is executed by echoing the command syntax into the `$GITHUB_ENV` file, which the GitHub Actions runner processes to update the environment for subsequent steps. Workflow commands provide a powerful way to manipulate the workflow environment and control the execution flow based on dynamic conditions or action outcomes.

Event Payloads and Webhooks

- Event payloads and webhooks in GitHub Actions provide a mechanism for workflows to react to a wide range of GitHub events, offering detailed context about the event that triggered the workflow. Payloads contain JSON-formatted data related to the event, such as the commit SHA, branch name, or issue details, enabling actions within the workflow to make informed decisions based on the event context.
- This workflow is triggered by a push to the main branch and includes a step that prints the entire event payload. By accessing `github.event`, the workflow can dynamically adjust its behavior, such as deploying to different environments based on the contents of the payload. Webhooks, on the other hand, allow external services to be notified of these events, enabling integrations that can act upon these GitHub activities, further extending the automation capabilities beyond GitHub's own ecosystem.

```
name: CI

on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v2
      - name: Print Push Event Payload
        run: echo "The event payload: ${ toJson(github.event) }"
```

Security Practices for GitHub Actions

```
permissions:
  contents: read
  issues: write

jobs:
  example:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
        with:
          node-version: '14'
      - run: npm install
      - run: npm test
```

- **Use secrets for sensitive data:** Store API keys, credentials, and other sensitive data as encrypted secrets, never hard-code them in your workflow files.
- **Limit permissions:** Use the permissions keyword to restrict the GitHub token's permissions, granting only the necessary access for the workflow to function.
- **Audit third-party actions:** Use actions from trusted sources. Pin actions to a specific SHA to avoid malicious modifications.
- **Regularly update actions:** Keep actions up to date to incorporate security patches and improvements.
- **Monitor and audit workflows:** Regularly review your workflows and run history for unusual activity or security warnings.

Testing GitHub Actions

Testing your GitHub Actions workflows and custom actions ensures they operate as intended, catching errors early in the development process. Key strategies include:

- **Unit Testing for Custom Actions:** Develop unit tests for your custom actions, particularly if they're complex or have critical functionality. Use testing frameworks appropriate for the action's language, such as Jest for JavaScript.
- **Workflow Runs on Feature Branches:** Test workflows by triggering them on feature branches before merging into your main branch. This allows you to make necessary adjustments without impacting the main workflow.
- **Act Tool for Local Testing:** Utilize tools like `nektos/act` to run your workflows locally, simulating GitHub Actions environment on your machine. This speeds up the testing process.
- **Use of Dummy Secrets for Testing:** Configure your workflows with dummy secrets to test secret usage scenarios, ensuring that the real secrets are secure.

Debugging GitHub Actions

Debugging GitHub Actions involves a strategic approach to identify and resolve issues within your workflows. One effective technique is to enable step-by-step logging to gain insights into the execution process of your actions. This can be done by setting the `ACTIONS_STEP_DEBUG` secret in your repository to true, which enables detailed logging for each step in your workflows. Additionally, using conditional execution with `if` statements can help isolate problems by selectively running parts of your workflow based on custom conditions.

Managing Workflow Runs

Managing workflow runs in GitHub Actions involves monitoring their execution, cancelling unnecessary or problematic runs, and rerunning workflows to address failures or test changes. GitHub provides a user interface for these tasks, accessible within the Actions tab of a repository. Monitoring is straightforward through this dashboard, where the status and results of each run are displayed.

To cancel a running workflow, you can use the UI to select a specific run and then use the "Cancel workflow" option. For rerunning workflows, GitHub offers the ability to rerun failed jobs or the entire workflow from the same interface, which is especially useful for addressing transient errors or after making fixes.

Notifications in GitHub Actions

Notifications in GitHub Actions provide updates on the status of workflow runs, helping teams stay informed about their automation processes. GitHub Actions supports several types of notifications, including email, web, and mobile notifications, configurable through the repository settings or workflow files.

For workflow-specific notifications, you can use the `on:` [status] events in your workflow file to trigger notifications upon certain outcomes (success, failure, cancellation). Additionally, integrating third-party notification services via marketplace actions allows for customized notification channels like Slack, Microsoft Teams, or SMS.

```
name: Notify on Failure
on:
  push:
    branches:
      - main
jobs:
  build-and-notify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Build
        run: exit 1 # Simulate a build failure
      - name: Notify Failure
        if: failure()
        uses: some/notification-action@v1
        with:
          message: 'Build failed!'
          token: ${ secrets.NOTIFICATION_TOKEN }
```

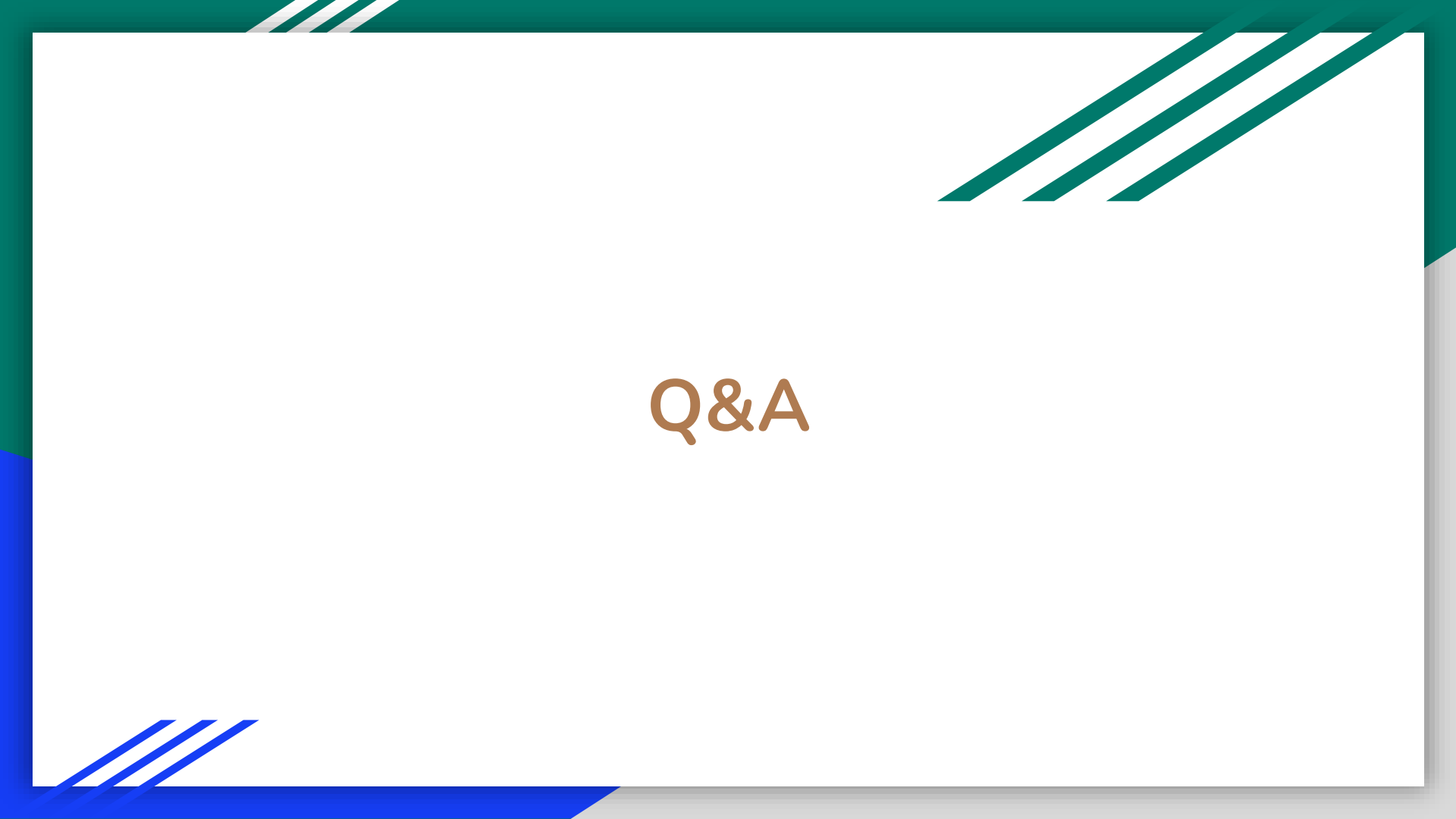
```
name: Build and Use Artifacts
on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Compile the project
        run: |
          mkdir build
          echo "Compiling project..." > build/output.txt
      - name: Upload artifacts
        uses: actions/upload-artifact@v2
        with:
          name: compiled-output
          path: ./build/

  use-artifacts:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Download artifacts
        uses: actions/download-artifact@v2
        with:
          name: compiled-output
      - name: Use the artifacts
        run: |
          ls -R
          cat compiled-output/output.txt
```

Artifacts and Logs

In GitHub Actions, artifacts are files generated during workflow runs, such as binaries, logs, or test results, which you can store and download after a job completes. Storing artifacts is crucial for debugging and allows for sharing data between jobs and after workflow completion. Accessing logs provides insight into the execution process, helping to troubleshoot failures or unexpected behavior.



Q&A